

Lesson 5

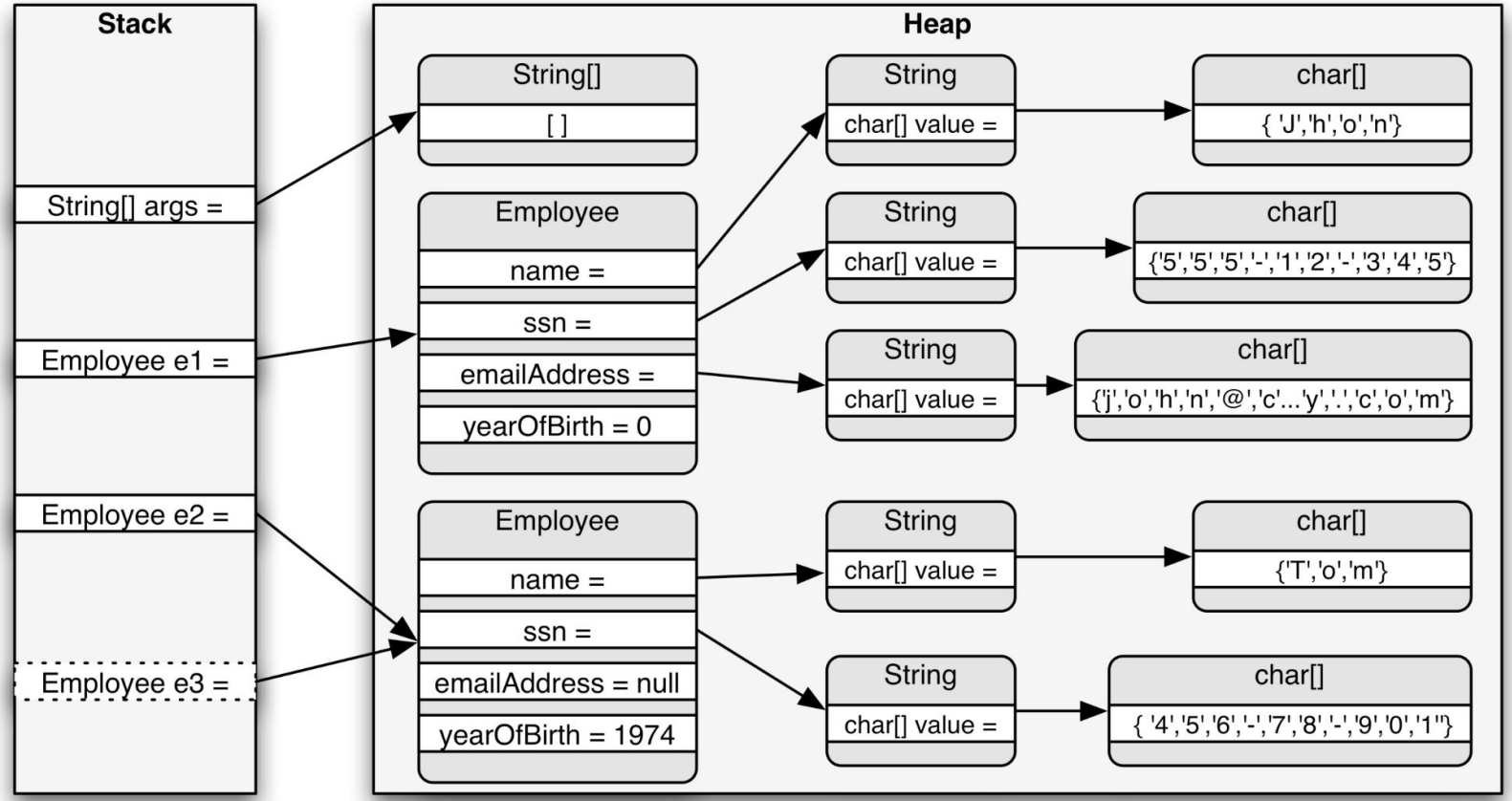
05.01.2026

```java
public class Task1 {
    public static void main(String[] args) {
        int a = 3;
        int b = a++ + ++a;
        System.out.println("a: " + a);
        System.out.println("b: " + b);
    }
}
```

```java
public class Task2 {
    public static void main(String[] args) {
        int a = 5;
        int b = 10;
        int c = a++ + --b + ++a - b--;
        System.out.println("a: " + a);
        System.out.println("b: " + b);
        System.out.println("c: " + c);
    }
}
```

```java
public class Task3 {
    public static void main(String[] args) {
        int num = -5;
        int absValue = (num < 0) ? -num : num;
        System.out.println("Absolute value: " + absValue);
    }
}
```

```java
public class Task4 {
    public static void main(String[] args) {
        int x = 10, y = 15;
        String result = (x > y) ? "x больше y" : ((x < y) ? "x меньше y" : "x равен y");
        System.out.println(result);
    }
}
```


Stack
String[] args =
Employee e1 =
Employee e2 =
Employee e3 =
Heap
String[]
[ ]
Employee
name =
ssn =
emailAddress =
yearOfBirth = 0
Employee
name =
ssn =
emailAddress = null
yearOfBirth = 1974
String
char[] value =
String
char[] value =
String
char[] value =
String
char[] value =
String
char[] value =
char[]
{ 'J','h','o','n'}
char[]
{'5','5','5','-','1','2','-','3','4','5'}
char[]
{'j','o','h','n','@','c'...'y','.','c','o','m'}
char[]
{'T','o','m'}
char[]
{ '4','5','6','-','7','8','-','9','0','1'}



Java variables do not contain the actual objects, they contain *references* to the objects.

- The actual objects are stored in an area of memory known as the *heap*.
- Local variables referencing those objects are stored on the stack.
- More than one variable can hold a reference to the same object.

Клас Object визначає метод clone(), що створює копію об'єкта. Якщо ви хочете, щоб екземпляр вашого класу можна було клонувати, необхідно перевизначити цей метод та реалізувати інтерфейс Cloneable. Інтерфейс Clonable – це інтерфейс-маркер, він не містить ні методів, ні змінних. Інтерфейси-маркер просто визначають поведінку класів.

Object.clone() викидає виняток CloneNotSupportedException при спробі клонувати об'єкт, що не реалізує інтерфейс Cloneable. Метод clone() у батьківському класі Object є protected, тому бажано перевизначити його як public. Реалізація за умовчанням методу Object.clone() виконує неповне/поверхневе (shallow) копіювання

```
public class Student implements Cloneable {

    @Override
    protected Object clone() throws CloneNotSupportedException {
        return super.clone();
    }
```



КЛОНУВАННЯ ОБ'ЄКТІВ У JAVA

Клонування – це створення копії об'єкта, яка має такі ж значення полів, але є окремим незалежним об'єктом у пам'яті.

НАВІЩО?

- Створення резервної копії об'єкта
- Безпечне модифікування копії
- Реалізація шаблонів (Prototype тощо)

РЕЗУЛЬТАТ:

Отримуємо новий об'єкт з однаковим станом, але іншим місцем у пам'яті.

1. ЩО ТАКЕ КЛОНУВАННЯ?



obj1 і obj2 – різні об'єкти (різні адреси в пам'яті), але мають однакові значення полів.

Зміни у копії не впливають на оригінал (для глибокого копіювання).

2. ЯК ПРАЦЮЄ CLONE В JAVA?

1. Клас повинен реалізувати інтерфейс `Cloneable`
2. Перевизначити метод `Object clone()`
3. Метод `clone()` створює копію об'єкта

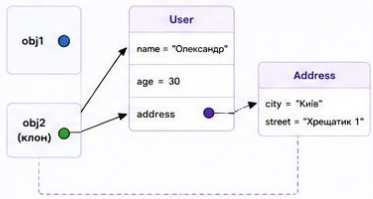
```
public class User implements Cloneable {
    private String name;
    private int age;
    private Address address; // складове поле

    @Override
    protected Object clone() throws CloneNotSupportedException {
        return super.clone();   // поверхове клонування
    }
}
```

Метод Object.clone() за замовчуванням виконує поверхове клонування (shallow copy).

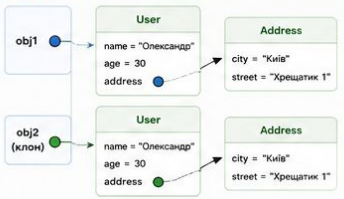
3. ПАМ'ЯТЬ: ПОВЕРХОВЕ (SHALLOW) vs ГЛИБОКЕ (DEEP) КЛОНУВАННЯ

ПОВЕРХОВЕ КЛОНУВАННЯ (shallow copy)



obj1 і obj2 посилаються на один і той самий об'єкт Address. Зміна address.city через obj2 вплине і на obj1!

ГЛИБОКЕ КЛОНУВАННЯ (deep copy)



obj1 і obj2 мають посилання на різні об'єкти Address. Зміни у копії не впливають на оригінал.

4. ПРИКЛАД КОДУ

```
class Address implements Cloneable {
    String city;
    String street;

    Address(String city, String street) {
        this.city = city;
        this.street = street;
    }

    @Override
    protected Object clone() throws CloneNotSupportedException {
        return super.clone();
    }
}
class User implements Cloneable {
    String name;
    int age;
    Address address;

    User(String name, int age, Address address) {
        this.name = name;
        this.age = age;
        this.address = address;
    }

    @Override
    protected Object clone() throws CloneNotSupportedException {
        return super.clone();   // поверхове копіювання
    }
    // Глибоке копіювання
    public User deepClone() throws CloneNotSupportedException {
        User cloned = (User) super.clone();
        cloned.address = (Address) this.address.clone();
        return cloned;
    }
}
```

```
public class Main {
    public static void main(String[] args) throws Exception {
        Address addr = new Address("Київ", "Хрещатик 1");
        User original = new User("Олександр", 30, addr);

        // Поверхове клонування
        User shallow = (User) original.clone();

        // Глибоке клонування
        User deep = original.deepClone();

        // Змінимо місто через shallow
        shallow.address.city = "Львів";

        System.out.println("Оригінал: " + original.address.city);
        System.out.println("Shallow:  " + shallow.address.city);
        System.out.println("Deep:     " + deep.address.city);
    }
}
```

РЕЗУЛЬТАТ:

```
Оригінал: Київ
Shallow:  Львів    // зміна вплинула на оригінал!
Deep:     Київ     // глибока копія не залежить
```



5. ВАЖЛИВО ЗНАТИ

- clone() у Object має модифікатор protected, тому викликається всередині класу або його нащадків.
- Якщо клас містить посилання на інші об'єкти, потрібно реалізувати глибоке клонування.
- Інтерфейс Cloneable не містить методів – це лише маркер.
- Якщо не перевизначити clone(), буде створено поверхову копію.
- Масиви вже підтримують клонування:

```
int[] arr = {1, 2, 3};
int[] copy = arr.clone();
```



6. АЛЬТЕРНАТИВИ КЛОНУВАННЮ

Конструктор копіювання

Створіть конструктор, який приймає об'єкт і копіює поля.

Builder / Factory / Prototype

Використовуйте патерн Prototype для створення копій.

Серіалізація

Серіалізуйте об'єкт у байти та десеріалізуйте назад (повне глибоке клонування).

КОЛИ ВИКОРИСТОВУВАТИ?

- Коли потрібно швидко отримати копію об'єкта.
- Коли об'єкт складний і конструктор копіювання незручний.
- У шаблонах проектування (Prototype).



ПІДСУМОК

Клонування дозволяє створювати копії об'єктів.
За замовчуванням – це поверхове копіювання.
Для повної незалежності потрібно реалізовувати глибоке копіювання.



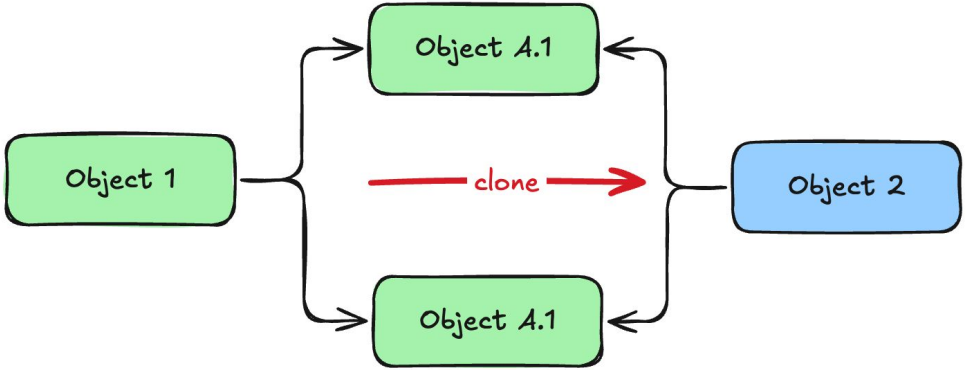
ЛЕГЕНДА

посилання на об'єкт у купі

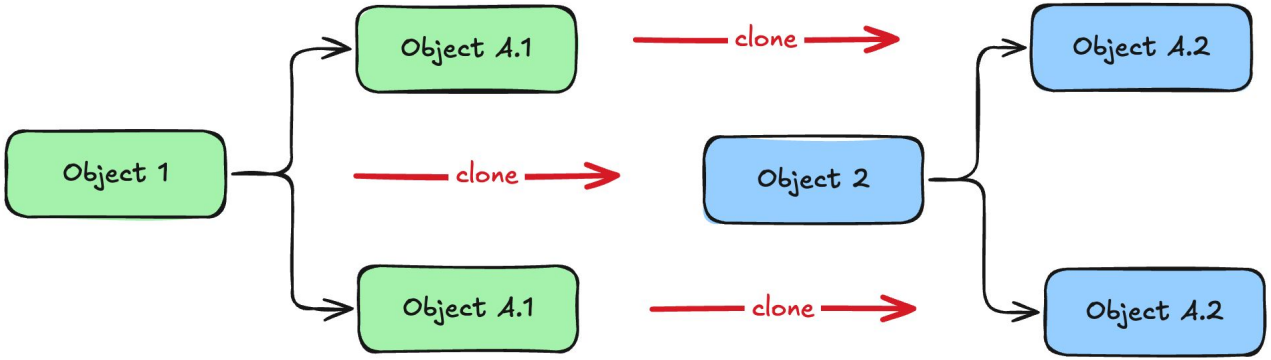
внутрішнє посилання всередині об'єкта



Shallow Cloning



Deep Cloning




this vs super

Чим this і super схожі

- І **this**, і **super** - це нестатичні змінні, відповідно їх не можна використовувати у статичному контексті, а це означає, що їх не можна використовувати у методі main. Це призведе до помилки під час компіляції "на нестатичну змінну цього не можна посилатися зі статичного контексту ". те саме станеться, якщо в методі main скористатися ключовим словом super.
- І **this**, і **super** можуть використовуватися всередині конструкторів для виклику інших конструкторів по ланцюжку, нпр., **this()** і **super()** викликають конструктор без аргументів спадкового та батьківського класів відповідно.
- Всередині конструктора this і super повинні стояти вище за всіх інших виразів, на початку, інакше компілятор видасть повідомлення про помилку. З чого випливає, що в одному конструкторі не може бути одночасно і this(), та super().

Відмінності у super та this

- Змінна this посилається на поточний екземпляр класу, в якому вона використовується, тоді як super - на екземпляр батьківського класу.
- Кожен конструктор за відсутності явних викликів інших конструкторів неявно викликає за допомогою super() конструктор без аргументів батьківського класу, при цьому у вас завжди залишається можливість явно викликати будь-який інший конструктор з допомогою або this(), або super().



this і super – ключові посилання в Java, які допомагають працювати з поточним об'єктом та батьківським класом.

THIS vs SUPER у Java

НАВІЩО?

- Уникати конфліктів імен
- Викликати потрібний конструктор або метод
- Писати зрозумілий та безпечний код

РЕЗУЛЬТАТ:

Чистий, гнучкий код і коректна робота з успадкуванням.

1. THIS – посилання на поточний об'єкт

this вказує на поточний екземпляр класу.

Поле класу	Розрізняє поле об'єкта від параметра з такою ж назвою.
Метод класу	Викликає інший метод цього ж класу.
Конструктор класу	Викликає інший конструктор цього ж класу.
Передача поточного об'єкта	Передає поточний об'єкт як аргумент у метод.

ПРИКЛАДИ

```
class Person {
   private String name;

   // 1) Поле класу
   Person(String name) {
       this.name = name; // this.name = поле об'єкта
   }

  // 2) Метод класу
  void setName(String name) {
       this.name = name;
   }

  // 3) Виклик іншого конструктора
  Person() {
       this("Unknown");
   }

  // 4) Передача об'єкта
  void printInfo() {
       print(this);
  }
  void print(Person p) {
    System.out.println("Name:" + p.name);
  }
}
```



2. SUPER – посилання на батьківський клас

super використовується в нащадку для роботи з членами батьківського класу.

Поле батьківського класу	Звертається до поля батьківського класу, якщо воно приховано (hidden).
Метод батьківського класу	Викликає перевизначений метод батьківського класу.
Конструктор батьківського класу	Викликає конструктор батьківського класу. Має бути першим у конструкторі нащадка.

ПРИКЛАДИ

```
class Animal {
  String name = "Тварина";

  Animal(String name) {
      this.name = name;
  }

  void speak() {
     System.out.println("Тварина видає звук");
  }
}

class Dog extends Animal {
  String name = "Собака"; // приховує поле батька

  Dog(String name) {
     super(name);       // 3) виклик конструктора батька
  }

  void show() {
     System.out.println("this.name = " + this.name);  // Собака
     System.out.println("super.name = " + super.name); // Тварина
  }

  @Override
  void speak() {
     super.speak(); // 2) виклик методу батька
     System.out.println("Гав-гав!");
  }
```



3. ЯК ЦЕ ПРАЦЮЄ В ПАМ'ЯТІ

У методі об'єкта нащадка можна використовувати:

- this – посилання на поточний об'єкт (екземпляр Dog)
- super – посилання на частину цього об'єкта, що належить батьківському класу (Animal)





ПРИКЛАД ВИКЛИКУ

```
Dog dog = new Dog("Барбос");
dog.show();
```

Вивід:

```
this.name = Собака
super.name = Барбос
```



4. ОСОБЛИВОСТІ ТА ПРАВИЛА

- this можна використовувати в будь-якому методі або конструкторі класу.
- this() / this(...) – виклик іншого конструктора цього ж класу. Має бути першим у конструкторі.
- super() / super(...) – виклик конструктора батьківського класу. **Має бути першим у конструкторі нащадка.**
- this і super не можна використовувати в статичних (static) методах або полях.



5. ПОШИРЕНІ ПОМИЛКИ

- Забули this – відбулося присвоєння параметра самому собі.
- Виклик super() не є першим у конструкторі.
- Спроба використати this або super у static контексті.



6. ПОРІВНЯННЯ THIS vs SUPER

Що	this	super
На що вказує	на поточний об'єкт	на батьківський клас (частину об'єкта)
Доступ	до членів поточного класу	до членів батьківського класу
Конструктор	this() – інший конструктор цього ж класу	super() – конструктор батьківського класу
Виклик метода	інший метод поточного класу	перевизначений метод батьківського класу
Випадки використання	усунення конфліктів імен, виклик своїх методів/конструкторів, передача себе як аргумент	доступ до прихованих членів батька, повторне використання логіки батька, ініціалізація батьківської частини об'єкта



КЛЮЧОВЕ СЛОВО

Використовуйте this і super правильно – і ваш код буде зрозумілим, безпечним та готовим до масштабування!



ПРИКЛАД РЕАЛЬНОГО СЦЕНАРІЮ

У банківській системі клас Account (базовий) містить загальні дані. Клас SavingsAccount (нащадок) використовує super для виклику логіки батька та this для роботи зі своїми даними.



Інтерфейс – це контракт, в рамках якого частини програми, найчастіше написані різними людьми, взаємодіють між собою та із зовнішніми додатками. Інтерфейси працюють з шарами сервісів, безпеки, DAO та і т.д. Це дозволяє створювати модульні конструкції, у яких для зміни одного елемента не потрібно чіпати решту.

```
package com.hillel;

public interface Say {

    void sayHello();

    default void sayGoodbye(){
        System.out.println("Goodbye ... ");
    }
}
```

У класі, що імплементує інтерфейс, повинні бути реалізовані всі передбачені інтерфейсом методи, за винятком методів замовчуванням (**default**).

Методи за замовчуванням вперше з'явилися в Java 8. Їх позначають модифікатором default. У нашому прикладі це метод sayGoodbye, реалізація якого прописано прямо в інтерфейсі. Дефолтні методи спочатку готові до використання, але при необхідності їх можна перевизначати в застосовуючи інтерфейс класи.

ІНТЕРФЕЙС (INTERFACE)

Контракт поведінки. Гнучкість. Розширюваність.

МЕТА

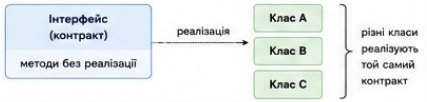
Інтерфейс визначає набір методів, які клас (реалізація) зобов'язується реалізувати.

КОЛИ ВИКОРИСТОВУВАТИ?

- Коли різні класи мають схожу поведінку
- Для досягнення поліморфізму
- Для слабкої зв'язаності компонентів
- Для тестування (мок-об'єкти)

1. ЩО ТАКЕ ІНТЕРФЕЙС?

Інтерфейс – це тип (контракт), який містить тільки підписи методів (без реалізації, до Java 8 – також без полів). Клас, що реалізує інтерфейс, повинен реалізувати всі його методи.



Переваги:

- Поліморфізм
- Гнучкість і розширюваність
- Незалежність від конкретних реалізацій
- Легкість тестування та підміни реалізацій

2. СИНТАКСИС

Оголошення інтерфейсу:

```
public interface MyInterface {
    void method1();          // абстрактний метод
    int method2(int a, int b);

    // з Java 8 – можна мати методи з реалізацією
    default void log(String msg) {
        System.out.println("LOG: " + msg);
    }

    // з Java 8 – статичні методи
    static int util(int a, int b) {
        return a + b;
    }

    // з Java 9 – приватні допоміжні методи
    private void helper() {
        // допоміжна логіка
    }
}
```

Реалізація інтерфейсу:

```
public class MyClass implements MyInterface {
    @Override
    public void method1() {
        // реалізація
    }

    @Override
    public int method2(int a, int b) {
        return a - b;
    }

    // можна перевизначити default метод
    @Override
    public void log(String msg) {
        System.out.println("MyClass: " + msg);
    }
}
```

Важливо!

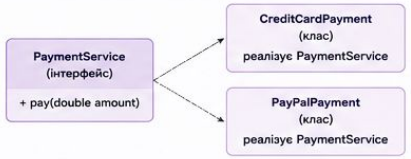
Клас, який реалізує інтерфейс, повинен реалізувати всі його абстрактні методи.

3. ПОЛІМОРФІЗМ ЧЕРЕЗ ІНТЕРФЕЙС

Посилання типу інтерфейсу може вказувати на об'єкти різних класів, які реалізують цей інтерфейс.

```
PaymentService service = new CreditCardPayment();
service.pay(100);  // Викличеться реалізація CreditCardPayment

service = new PayPalPayment();
service.pay(200);  // Викличеться реалізація PayPalPayment
```



4. ПРИКЛАД

Інтерфейс:

```
public interface Animal {
    void sound();
    void move();
}
```

Реалізації:

```
public class Dog implements Animal {
    @Override
    public void sound() {
        System.out.println("Гав-гав");
    }

    @Override
    public void move() {
        System.out.println("Біжить");
    }
}
```

```
public class Cat implements Animal {
    @Override
    public void sound() {
        System.out.println("Мяв-мяв");
    }

    @Override
    public void move() {
        System.out.println("Йде тихо");
    }
}
```

Використання:

```
Animal a1 = new Dog();
Animal a2 = new Cat();
a1.sound();  // Гав-гав
a2.move();   // Йде тихо
```

Ми працюємо з інтерфейсом Animal, тому код не залежить від конкретного класу.

5. ІНТЕРФЕЙС vs АБСТРАКТНИЙ КЛАС

Критерій	Інтерфейс (Interface)	Абстрактний клас (Abstract Class)
Призначення	Контракт поведінки	Спільний базовий клас
Методи (до Java 8)	Тільки абстрактні (підписи без тіла)	Можуть бути абстрактні і з реалізацією
Методи (з Java 8+)	Можуть бути default, static, private	-
Поля	Тільки public static final (константи)	Можуть бути будь-які
Конструктор	Немає	Може бути
Наслідування	Клас може реалізувати кілька інтерфейсів	Клас може наслідувати лише один клас
Використання	Опис можливостей (що може робити)	Опис сутності (що є чимось)

6. МНОЖИННЕ НАСЛІДУВАННЯ ІНТЕРФЕЙСІВ

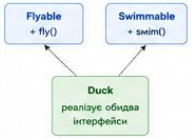
Приклад:

```
public interface Flyable {
    void fly();
}

public interface Swimmable {
    void swim();
}

public class Duck implements Flyable, Swimmable {
    @Override
    public void fly() { System.out.println("Летить"); }

    @Override
    public void swim() { System.out.println("Плаває"); }
}
```



Чому це важливо?

Клас може мати поведінки з різних джерел, не створюючи глибокої ієрархії успадкування.

7. ІНТЕРФЕЙС ЯК ТИП ДЛЯ ЗАЛЕЖНОСТЕЙ

Програмуємо на рівні інтерфейсів, а не реалізацій (принцип Dependency Inversion).

Погано (залежність від реалізації):

```
class OrderService {
    private EmailSender sender = new EmailSender();
    // ....
}
```

Добре (залежність від інтерфейсу):

```
interface Sender {
    void send(String to, String msg);
}

class OrderService {
    private Sender sender;
    public OrderService(Sender sender) {
        this.sender = sender;
    }
}
```

- Легко змінювати реалізацію
- Легко тестувати (mock)
- Менше зв'язності – краща архітектура

8. DEFAULT МЕТОДИ (Java 8+)

У інтерфейсі:

```
public interface Logger {
    void log(String msg);

    default void info(String msg) {
        log("INFO: " + msg);
    }

    default void error(String msg) {
        log("ERROR: " + msg);
    }
}
```

Використання:

```
public class ConsoleLogger implements Logger {
    @Override
    public void log(String msg) {
        System.out.println(msg);
    }
}

Logger logger = new ConsoleLogger();
logger.info("Система стартувала");
// INFO: Система стартувала
```

Default методи дозволяють додавати нову функціональність в інтерфейс без поломки існуючих реалізацій.

9. КЛЮЧОВІ ТЕРМІНИ

Інтерфейс	Контракт (набір методів), який клас повинен реалізувати.
Реалізація	Клас, який реалізує інтерфейс і надає тіла методів.
Поліморфізм	Можливість використовувати різні реалізації через один інтерфейс.
Default метод	Метод в інтерфейсі з реалізацією (з Java 8+).
Static метод	Метод інтерфейсу, що належить типу інтерфейсу (з Java 8+).
Множинна реалізація	Клас може реалізувати кілька інтерфейсів одночасно.
Абстракція	Приховування деталей реалізації, показується лише контракт.
Слабка зв'язаність	Компоненти залежать від абстракцій, а не від конкретних класів.

ГОЛОВНЕ

- Інтерфейс описує ЩО робити, а не ЯК.
- Класи реалізують інтерфейси і визначають ЯК.
- Використовуйте інтерфейси для гнучкості, тестованості та підтримуваності коду.

ПРАВИЛО

Програмуйте на рівні інтерфейсів, а не реалізацій!

ІНТЕРФЕЙС vs НАСЛІДУВАННЯ

Два різні механізми повторного використання коду

ГОЛОВНА ІДЕЯ

Наслідування (extends) – це "є-а" відношення (це тип відношення).
Інтерфейс (implements) – це "може робити" (це контракт поведінки).

КОЛИ ЩО ВИКОРИСТОВУВАТИ?

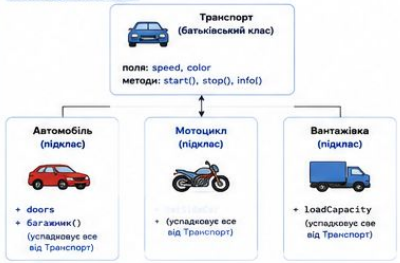
- Наслідування – коли є природна ієрархія "є-а".
- Інтерфейс – коли потрібен контракт поведінки, незалежно від ієрархії класів.

1. НАСЛІДУВАННЯ (Inheritance)

Що це?

Механізм, коли один клас (підклас) успадковує поля і методи іншого класу (батьківського класу). Використовується для створення ієрархії "є-а".

ПРИКЛАД: ТРАНСПОРТ



Ключове слово: `extends`

```
clans Transport {
    int speed;
    String color;

    void start()  { System.out.println("Запуск..."); }
    void stop()   { System.out.println("Зупинка..."); }
    void info()   { System.out.println("Транспорт"); }
}

class Car extends Transport {
  int doors;
  void trunk()  { System.out.println("Багажник відкрито"); }
}

class Truck extends Transport {
  int loadCapacity;
}

Car car = new Car();
car.speed = 120;      // успадковане поле
car.start();          // успадкований метод
car.trunk();          // свій метод
```

Особливості наслідування

- Підклас успадковує стан (поля) і поведінку (методи).
- Єдиний клас не може успадковувати кілька класів.
- Підклас може перевизначати методи батьківського класу (@Override).
- Створює тісний зв'язок між класами.
- Використовуйте, коли є чітка ієрархія "є-а".

Тип відношення

"Є-А" (is-a)

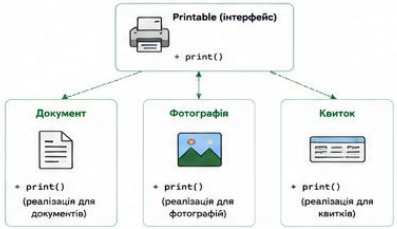
Автомобіль є Транспортом
Car is-a Transport

2. ІНТЕРФЕЙС (Interface)

Що це?

Інтерфейс – це контракт (набір методів без реалізації), який клас зобов'язується реалізувати. Використовується для визначення поведінки, незалежно від ієрархії класів.

ПРИКЛАД: ПРИСТРОЇ, ЩО МОЖУТЬ ДРУКУВАТИ



Ключове слово: `implements`

```
interface Printable {
    void print();    // контракт без реалізації
}

class Document implements Printable {
    @Override
    public void print() {
        System.out.println("Друк документу");
    }
}

class Photo implements Printable {
    @Override
    public void print() {
        System.out.println("Друк фотографії");
    }
}

Printable p1 = new Document();
Printable p2 = new Photo();
p1.print();     // Друк документу
p2.print();     // Друк фотографії
```

Особливості інтерфейсу

- Містить лише оголошення методів (до Java 8 – лише абстрактні методи).
- Клас може реалізовувати багато інтерфейсів.
- Дозволяє слабке зв'язування між класами.
- Відповідає принципу розділення інтерфейсів (ISP).
- Використовуйте, коли класи різні, але мають спільну поведінку.

Тип відношення

"МОЖЕ РОБИТИ" (can-do)

Документ може друкувати
Document can-do Print

3. ПОРІВНЯННЯ

Критерій	Наслідування (extends)	Інтерфейс (implements)
Тип відношення	**"Є-А" (is-a)**	**"Може робити" (can-do)**
Призначення	Повторне використання коду, побудова ієрархії	Визначення контракту поведінки
Ключове слово	`extends`	`implements`
Успадковуються поля	Так	Ні (лише константи)
Успадковуються методи (реалізація)	Так	Ні (тільки сигнатури, реалізація в класі)
Кількість батьків	Один клас	Багато інтерфейсів
Доступ модифікатора	protected, public	public (методи), public static final (поля)
Коли використовувати	Коли є природна ієрархія	Коли є спільна поведінка для різних класів
Приклад	`Car extends Transport`	`Document implements Printable`

4. КОМБІНАЦІЯ: ІНТЕРФЕЙСИ + НАСЛІДУВАННЯ

Клас може наслідувати інший клас і реалізовувати один або декілька інтерфейсів одночасно.

```
interface Movable {
    void move();
}

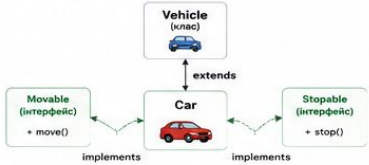
interface Stopable {
    void stop();
}

class Vehicle {
    int speed;

class Car extends Vehicle implements Movable, Stopable {

    @Override
    public void move()  { System.out.println("Їду..."); }

    @Override
    public void stop()  { System.out.println("Зупиняюсь"); }
}
```



5. ПРИКЛАДИ З РЕАЛЬНОГО ЖИТТЯ

Наслідування (is-a)	Інтерфейс (can-do)
Собака є Твариною	Принтером може друкувати Документ, Фото, Квиток
Квадрат є Прямокутником	Літати можуть Птах, Літак, Дрон
Легковий автомобіль є Автомобілем	Зберігати можуть Файл, БазаДаних, ХмарнеСховище

6. ПЕРЕВАГИ ТА НЕДОЛІКИ

Наслідування	Інтерфейс
✅ Просте повторне використання коду	✅ Гнучкість і слабке зв'язування
✅ Є природним для ієрархій	✅ Підтримка множинного наслідування типу
✅ Можна перевизначати методи	✅ Легко додаються нові реалізації
❌ Тісний зв'язок між класами	❌ Не містить стану (лише константи)
❌ Один батьківський клас	❌ Потрібно реалізовувати всі методи
❌ Проблема "крихкої бази" (fragile base class)	❌ Більше коду (потрібна реалізація)

7. КОЛИ ВИКОРИСТОВУВАТИ?

Використовуйте НАСЛІДУВАННЯ, якщо:

- ✅ Є чітка ієрархія "є-а".
- ✅ Потрібно перевикористати код.
- ✅ Підклас дійсно є спеціалізацією батька.

Використовуйте ІНТЕРФЕЙС, якщо:

- ✅ Потрібен контракт поведінки.
- ✅ Класи не пов'язані ієрархією.
- ✅ Очікуєте розширення системи.

ЗОЛОТЕ ПРАВИЛО

Спочатку думайте про ІНТЕРФЕЙСИ – вони роблять код гнучким. Використовуйте НАСЛІДУВАННЯ – тільки коли справді є відношення "є-а".

ПАМ'ЯТАЙТЕ

Інтерфейс описує, **ЩО** клас може робити.
Наслідування описує, **КИМ** клас є.

Якщо інтерфейс має лише один абстрактний метод, перед нами функціональний інтерфейс. Його прийнято позначати інструкцією **@FunctionalInterface**, яка вказує компілятору, що при виявленні другого абстрактного методу у цьому інтерфейсі потрібно повідомити про помилку. Стандартних (default) методів у інтерфейсу може бути безліч – у тому числа що належать класу java.lang.Object

```
@FunctionalInterface
public interface Developer {

    boolean isDeveloper();

}
```

ФУНКЦІОНАЛЬНИЙ ІНТЕРФЕЙС (Functional Interface)

Основи, приклади та використання в Java 8+

Ідея:
Функціональний інтерфейс містить рівно один абстрактний метод. Це дозволяє використовувати лямбда-вирази та методи-посилання.

ПЕРЕВАГИ
- Менше коду
- Гнучкість і виразність
- Підтримка функціонального стилю програмування
- Легко поєднувати та передавати поведінку як дані

1. ЩО ТАКЕ ФУНКЦІОНАЛЬНИЙ ІНТЕРФЕЙС?

Інтерфейс, який має тільки один абстрактний метод (метод без тіла реалізації).
Може мати будь-яку кількість:

- методів за замовчуванням (default)
- статичних методів
- методів з реалізацією (private)
- перевизначених методів Object (toString, equals, hashCode)

Такі інтерфейси позначаються анотацією @FunctionalInterface (не є обов'язковою, але рекомендована).

2. ПРИКЛАД СТВОРЕННЯ

```
@FunctionalInterface
interface MyFunctionalInterface {
   void execute(String msg);   // єдиний абстрактний метод

   // default метод
   default void info() {
      System.out.println("Це default метод");
   }

   // static метод
   static void helper() {
      System.out.println("Це static метод");
   }
}
```

Якщо додати ще один абстрактний метод – інтерфейс більше не буде функціональним.

3. ЛЯМБДА-ВИРАЗ

```
MyFunctionalInterface action = (msg) -> {
   System.out.println("Повідомлення: " + msg);
};

action.execute("Привіт!");   // Повідомлення: Привіт!
action.info();               // Це default метод
MyFunctionalInterface.helper(); // Це static метод
```

Як працює:
Лямбда-вираз реалізує єдиний абстрактний метод інтерфейсу – тобто підставляє реалізацію.

4. МЕТОДИ-ПОСИЛАННЯ

```
class Printer {
    public void printMessage(String msg) {
        System.out.println("Друк: " + msg);
    }
}

Printer printer = new Printer();
MyFunctionalInterface action = printer::printMessage;
action.execute("Hello");
```

Типи методів-посилань:

- `obj::instanceMethod` – посилання на метод об'єкта
- `ClassName::staticMethod` – посилання на статичний метод
- `ClassName::new` – посилання на конструктор

5. ПРИКЛАДИ ВБУДОВАНИХ ФУНКЦІОНАЛЬНИХ ІНТЕРФЕЙСІВ (java.util.function)

Інтерфейс	Опис	Абстрактний метод	Приклад використання
`Predicate<T>`	Приймає T, повертає boolean	`boolean test(T t)`	`s -> s.length() > 3`
`Function<T, R>`	Приймає T, повертає R	`R apply(T t)`	`s -> s.toUpperCase()`
`Consumer<T>`	Приймає T, нічого не повертає	`void accept(T t)`	`s -> System.out.println(s)`
`Supplier<T>`	Не приймає аргументів, повертає T	`T get()`	`() -> new Random().nextInt()`
`UnaryOperator<T>`	Приймає T, повертає T	`T apply(T t)`	`x -> x * x`
`BinaryOperator<T>`	Приймає два T, повертає T	`T apply(T t1, T t2)`	`(a, b) -> a + b`

6. ВИКОРИСТАННЯ З STREAM API

```
List<String> names = List.of("Іван", "Петро", "Оля", "Сергій");

// Всі імена довші за 3 символи, у верхньому регістрі, відсортовані
names.stream()
   .filter(name -> name.length() > 3)      // Prodicate
   .map(String::toUpperCase)               // Function
   .sorted()                               // Comparator
   .forEach(System.out::println);          // Consumer

// Рахуємо кількість імен, що починаються на "П"
long count = names.stream()
   .filter(n -> n.startsWith("П"))
   .count();
System.out.println( "Кількість: " + count);
```

Stream

7. ПЕРЕВІРКА НА ФУНКЦІОНАЛЬНІСТЬ

Інтерфейс є функціональним, якщо:

- рівно один абстрактний метод
- можна анотувати @FunctionalInterface
- компілятор дозволяє використання лямбда-виразу
- не є підтипом інших функціональних інтерфейсів з конфліктними методами

```
@FunctionalInterface
interface WrongInterface {
   void m1();
   void m2();    // ✗ ще один абстрактний метод
}
```

Не буде компілюватися з лямбда-виразом!

8. ПОШИРЕНІ СЦЕНАРІЇ

Фільтрація	`list.stream().filter(x -> x > 10)`
Перетворення	`list.stream().map(x -> x * 2)`
Передача поведінки як параметр	`process(() -> System.out.println("Готово!"));`
Асинхронні дії	`executon.submit(() -> doWork());`

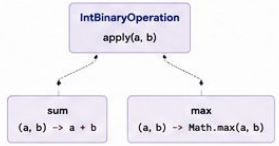
9. ВЛАСНИЙ ФУНКЦІОНАЛЬНИЙ ІНТЕРФЕЙС

Приклад: інтерфейс для операції з двома числами

```
@FunctionalInterface
interface IntBinaryOperation {
   int apply(int a, int b);
}

IntBinaryOperation sum = (a, b) -> a + b;
IntBinaryOperation max = (a, b) -> Math.max(a, b);

System.out.println(sum.apply(3, 5));   // 8
System.out.println(max.apply(3, 5));   // 5
```



10. КОРОТКО

- Функціональний інтерфейс = 1 абстрактний метод
- Підтримує лямбда-вирази та методи-посилання
- Є основою функціонального програмування в Java
- Широко використовується в Stream API, колекціях, багатопоточності та інших фреймворках

λ

КЛЮЧОВІ ТЕРМІНИ

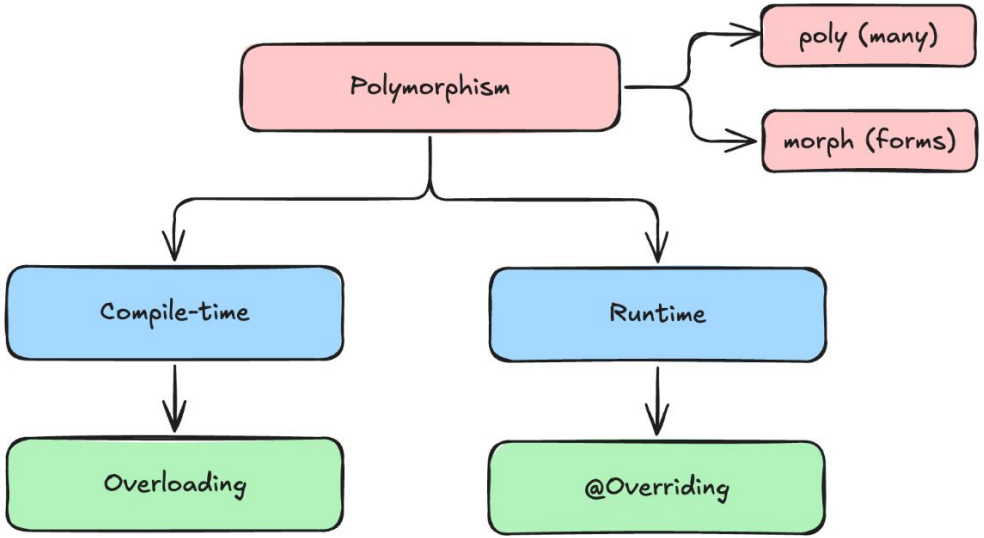
Абстрактний метод – метод без тіла реалізації.
Default метод – метод з реалізацією в інтерфейсі (default).
Лямбда-вираз – анонімна функція для реалізації методу.
Метод-посилання – скорочений запис виклику існуючого методу.
SAM (Single Abstract Method) – англ. термін для функціонального інтерфейсу.
@FunctionalInterface – анотація для позначення функціонального інтерфейсу.

ЗОЛОТЕ ПРАВИЛО
Якщо інтерфейс має одну задачу – зроби його функціональним!

Інтерфейси та поліморфізм.

У Java поліморфізм можна реалізувати через:

успадкування - з перевизначенням параметрів та методів базового класу;
абстрактні класи - шаблони для роздільної реалізації у різних класах;
інтерфейси – для імплементації класами.




```java
// Parent Class
class Parent {
    // Overloaded method (compile-time polymorphism)
    public void func() {
        System.out.println("Parent.func()");
    }

    // Overloaded method (same name, different parameter)
    public void func(int a) {
        System.out.println("Parent.func(int): " + a);
    }
}

// Child Class
class Child extends Parent {
    // Overrides Parent.func(int) (runtime polymorphism)
    @Override
    public void func(int a) {
        System.out.println("Child.func(int): " + a);
    }
}
```

Крім звичайних класів Java є **абстрактні класи**. Абстрактний клас схожий на клас. В абстрактному класі також можна визначити поля та методи, водночас не можна створити об'єкт або екземпляр абстрактного класу. Абстрактні класи покликані надавати базовий функціонал для класів спадкоємців. А похідні класи вже реалізують цей функціонал.

```
public abstract class Human {

    abstract void see();

    public void talk(String str){
        System.out.println(str);
    }

    public void hear(String str){
        System.out.println(str);
    }
}
```

- Інтерфейс описує лише поведінку. Він не має стану. А у абстрактного класу стан є: він описує і те, й інше.
- Абстрактний клас пов'язує між собою та об'єднує класи, що мають дуже близький зв'язок. У той же час, той самий інтерфейс можуть реалізувати класи, які взагалі немає нічого спільного.
- Класи можуть реалізовувати скільки завгодно інтерфейсів, але успадковуватися можна лише від одного класу

Ідея
Абстрактний клас не можна створювати. Він призначений для наслідування. Він може містити як реалізовані методи, так і абстрактні (без реалізації).

АБСТРАКТНИЙ КЛАС (abstract class)

Шаблон для інших класів. Містить спільний код і контракт поведінки.

КОЛИ ВИКОРИСТОВУВАТИ?
- ✅ Коли є спільний стан і поведінка для всіх підкласів
- ✅ Коли потрібно частково реалізувати функціональність
- ✅ Коли бажаємо задати "шаблон" для нащадків
- ✅ Коли потрібен спільний конструктор і поля

1. ЩО ТАКЕ АБСТРАКТНИЙ КЛАС?

Абстрактний клас (abstract class) — це клас, який оголошується з ключовим словом **abstract** і не може бути інстанційований (створений). Він служить базовим класом для інших класів.

```
abstract class Animal {
    // ...
}

Animal a = new Animal();  // ❌ Помилка компіляції
```

2. СИНТАКСИС ТА ПРИКЛАД

```
abstract class Animal {
    protected String name;

    // Конструктор
    public Animal(String name) {
        this.name = name;
    }

    // Звичайний метод з реалізацією
    public void sleep() {
        System.out.println(name + " спить");
    }

    // Абстрактний метод (без тіла)
    public abstract void makeSound();
}
```

✅ Може містити:
- поля
- конструктори
- звичайні методи
- абстрактні методи

❌ Не може:
- створювати об'єкти через new

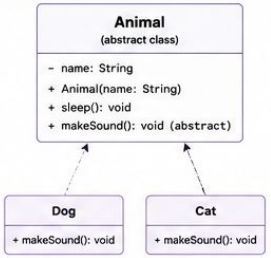
3. НАСЛІДУВАННЯ АБСТРАКТНОГО КЛАСУ

```
class Dog extends Animal {
    public Dog(String name) {
        super(name);
    }

    @Override
    public void makeSound() {
        System.out.println(name + " гав-гав");
    }
}

class Cat extends Animal {
    public Cat(String name) {
        super(name);
    }

    @Override
    public void makeSound() {
        System.out.println(name + " няв-няв");
    }
}
```



4. ПРИКЛАД ВИКОРИСТАННЯ

```
public class Main {
    public static void main(String[] args) {
        Animal a1 = new Dog("Бобік");   // Поліморфізм
        Animal a2 = new Cat("Мурка");

        a1.makeSound();    // Бобік гав-гав
        a2.makeSound();    // Мурка няв-няв
        a1.sleep();        // Бобік спить
    }
}
```

ℹ Ми працюємо через тип базового класу (Animal), але виконуються методи конкретних підкласів.



5. ПРАВИЛА

- ✅ Абстрактний клас оголошується з ключовим словом **abstract**.
- ✅ Якщо клас містить хоча б один абстрактний метод, то сам клас повинен бути **abstract**.
- ✅ Абстрактний метод не має тіла і оголошується через **abstract**.
- ✅ Клас-нащадок повинен реалізувати всі абстрактні методи, інакше він теж має бути абстрактним.
- ✅ Абстрактний клас може мати конструктор.

⚠ Не можна створити об'єкт абстрактного класу:
`Animal a = new Animal();  // ❌ Помилка компіляції`

6. АБСТРАКТНИЙ КЛАС vs ІНТЕРФЕЙС

	Абстрактний клас	Інтерфейс
Ключове слово	`abstract class`	`interface`
Методи	Абстрактні + звичайні	З Java 8: default, static, з Java 9: private
Поля	Мають стан (поля)	Усі поля public static final (константи)
Конструктор	Є	Немає
Наслідування	`extends` (тільки один клас)	`implements` (можна кілька)
Призначення	Спільна реалізація та стан	Контракт поведінки
Доступ	Будь-який модифікатор доступу	public (за замовчуванням)

7. АБСТРАКТНИЙ КЛАС МОЖЕ БУТИ АБСТРАКТНИМ УСПАДКОМ

```
abstract class Vehicle {
    protected int speed;
    public abstract void start();
}

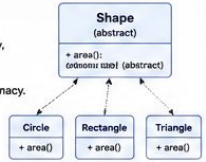
// Підклас теж абстрактний, бо не реалізує start()
abstract class Car extends Vehicle {
    public void stop() {
        System.out.println("Авто зупинилося");
    }
}
```

ℹ Клас Car не реалізує start(), тому він теж abstract. Конкретний підклас має реалізувати його.



8. КОЛИ ВИКОРИСТОВУАТИ?

- Коли є спільний код і стан для групи класів.
- Коли потрібно частково реалізувати поведінку, а решту залишити нащадкам.
- Коли потрібно надати конструктор базового класу.
- Коли важлива ієрархія «є-типом» (is-a).



9. КЛЮЧОВІ ТЕРМІНИ

Абстрактний клас	Клас, який не можна інстанціювати.
Абстрактний метод	Метод без реалізації (тільки підпис).
Конкретний клас	Клас, який можна інстанціювати.
Поліморфізм	Використання посилання базового типу для роботи з об'єктами підкласів.
Шаблон	Абстрактний клас задає структуру поведінки, яку реалізують нащадки.

ЗОЛОТЕ ПРАВИЛО
Використовуйте абстрактний клас, коли потрібно поділитися кодом та станом між спорідненими класами і надати часткову реалізацію поведінки.

ПАМ'ЯТАЙТЕ
Абстрактний клас — це каркас (template), а не готовий об'єкт. Реалізацію деталей виконують підкласи.

Abstract class	Interface
1) Abstract class can **have abstract and non-abstract** methods.	Interface can have **only abstract** methods. Since Java 8, it can have **default and static methods** also.
2) Abstract class **doesn't support multiple inheritance**.	Interface **supports multiple inheritance**.
3) Abstract class **can have final, non-final, static and non-static variables**.	Interface has **only static and final variables**.
4) Abstract class **can provide the implementation of interface**.	Interface **can't provide the implementation of abstract class**.
5) The **abstract keyword** is used to declare abstract class.	The **interface keyword** is used to declare interface.
6) An **abstract class** can extend another Java class and implement multiple Java interfaces.	An **interface** can extend another Java interface only.
7) An **abstract class** can be extended using keyword "extends".	An **interface** can be implemented using keyword "implements".
8) A Java **abstract class** can have class members like private, protected, etc.	Members of a Java interface are public by default.